



Stock Exchange

CEBP 17.01.2025

—

Deployment Scenario

VCARB

Popovici Andrei-Cristian

Pipa-Bulz Mircea-Gabriel

Pop Alessia-Maria

Linteş Claudiu Constantin

Overview

This document describes the deployment process of our web application Stock Exchange. In this document we will be describing how the system components are deployed using containerization and cloud infrastructure that we have chosen, and the tools and architecture.

System Overview

- Frontend
 - Angular Web Application
- Backend
 - Spring Boot REST API - Stock management and transaction handling
 - ASP.NET Core REST API - Users management and Banking operations
- Databases
 - MySQL
 - SQL Server

The ASP.NET Core API has a one-way communication with the Spring Boot API to retrieve and process stock related data while the frontend communicates directly with both API's. Each backend service uses its own database.

Deployment Approach

The System is deployed using containerization and cloud infrastructure. The chosen tools for this process are Docker for packaging each system component into an isolated container. AWS will be used as the provider for the cloud environment where the containers will be deployed and executed.

Docker

Docker is used to containerize each component of the system frontend and both Spring and .NET API's. They will be packaged in individual Docker Images. This approach ensures that all required dependencies and runtime configurations are included within each container, allowing the application to run consistently across different environments.

Why Docker?

Docker containers wrap up a piece of software in a complete filesystem that contains everything it needs to run: code, runtime, system tools, system libraries – anything you can install on a server. This guarantees that it will always run the same, regardless of the environment it is



running in. It also enables easier updates, as new versions of a component can be deployed by replacing the running container with a new one based on an updated Docker image.

AWS

We have chosen Amazon Web Services as our cloud platform due to its scalability, reliability, and widespread industry adoption. The Docker containers are deployed within the AWS cloud environment, where they can run on virtualized infrastructure Virtual Servers (Instances) such as EC2.

Why AWS?

AWS provides the necessary networking, compute resources, and security mechanisms required to expose the backend services to clients while isolating internal components such as databases. This allows the system to be accessed over the internet while keeping sensitive data protected within the cloud environment.

Another reason for choosing AWS is the experience of the team working with the platform.

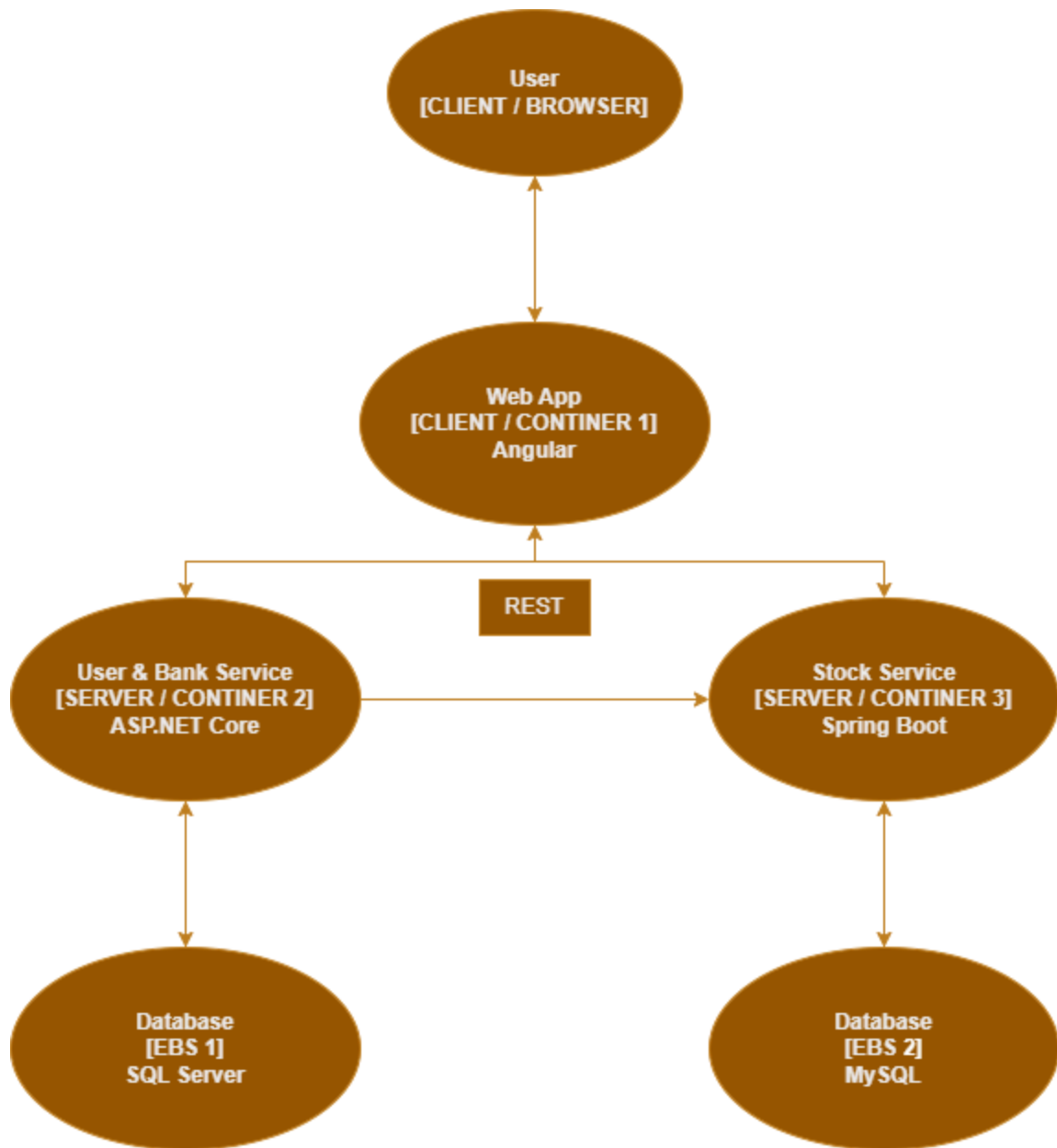
Integration

Containers can be deployed on AWS compute resources and communicate with each other through internal networking provided by the cloud infrastructure.

Configuration values such as database connection strings and service endpoints are supplied to the containers through environment variables.

The chosen deployment approach is intentionally kept simple and suitable for the scope of the project. Advanced container orchestration platforms such as Kubernetes are not used, mainly because orchestration or automatic scaling mechanisms would add unnecessary complexity to the deployment process.

Deployment diagram



Deployment Description

Client Interaction

End users access the system through a web browser. The frontend application is delivered to the client and serves as the main entry point to the system. User actions in the browser result in HTTP or HTTPS requests being sent to the backend services the requests are processed and appropriate responses are returned.

Frontend Deployment

The Angular frontend is packaged as a Docker container and deployed on an Amazon EC2 instance. The container hosts the frontend application and serves static web content to users.

The frontend communicates with the backend services using RESTful HTTP requests. Service endpoint URLs are provided to the frontend container via environment variables. Cross-Origin Resource Sharing (CORS) is configured on the backend services to allow requests from the frontend domain.

Backend Deployment

The backend consists of two independent services: a Spring Boot REST API and an ASP.NET Core REST API. Each backend service is packaged as a separate Docker image and deployed as a container on an Amazon EC2 instance. Both backend containers expose REST endpoints that are accessed by the frontend application. The ASP.NET Core service communicates with the Spring Boot service through internal REST calls to retrieve and process stock-related information ensuring proper transaction handling.

Database Deployment

Each backend service uses its own database, which is deployed separately from the application containers.

The Spring Boot service uses a MySQL database, while the ASP.NET Core service uses a SQL Server database. The database services run on EC2 instances with persistent storage provided by Amazon Elastic Block Store (EBS).

Configuration and Secrets Management

Configuration values such as database connection URLs, credentials, and service endpoints are provided to the Docker containers through environment variables.



Sensitive information, including database credentials, is not embedded within the Docker images.

Scalability and Maintainability

The use of Docker containers enables basic scalability and maintainability of the system. Although the system does not use advanced orchestration tools such as Kubernetes the containers can be restarted or replaced if failures occur, and updated versions of services can be deployed by rebuilding and redeploying the corresponding Docker images.